

Bachelor-Projekt

Sammeldokumentation der Gruppenarbeiten

Zwischenbericht

Hochschule Bremen

I.S. Medieninformatik

Modul: Bachelor-Projekt

Semester: Sommersemester 2026

Dozierende: Prof. Dr. Volker Paelke
Andreas Lochwitz

30. Juli 2026

Inhaltsverzeichnis

1	Projektname	3
1.1	Projektbeschreibung	3
1.2	Aufgabeneinteilung	3
1.3	Bisheriger Stand	3
1.4	Probleme	3
1.5	Nächste Schritte	3
2	OCMExplore	4
2.1	Problem	4
2.2	Idee	4
2.3	Projekt	5
2.4	Planung	5
2.5	Aufgabenteilung	6
2.6	Herausforderungen	6
2.7	Fazit	6
3	Schätze der Vergangenheit	8
3.1	Einführung	8
3.2	Konzept	9
3.3	Anforderungen	12
3.4	Entwicklungsprozess	14
3.5	Prototyp	15
3.5.1	Kippschalter-Level	15
3.5.2	Apple II Zusammenbau-Level	16
3.5.3	Quiz-Level	17
3.5.4	Code-Level	18
3.5.5	Technische Dokumentation	19
3.6	Aufgabenteilung	26
3.7	Resümee und Ausblick	26

4	Launcher App	28
4.1	Problem	28
4.2	Idee	28
4.3	Projekt	28
4.4	Aufgabeneinteilung	29
4.5	Weitere Überlegungen	29
4.5.1	Modularität	29
4.5.2	Vor-Ort Umsetzung	29
4.5.3	Design	29
4.6	Fazit	29

Kapitel 1

Projektname

1.1 Projektbeschreibung

Beschreibt euer Projekt.

1.2 Aufgabeneinteilung

Wer hat was gemacht?

1.3 Bisheriger Stand

Was wurde bisher umgesetzt?

1.4 Probleme

Welche Schwierigkeiten gab es?

1.5 Nächste Schritte

Was ist bis zur nächsten Abgabe geplant?

Kapitel 2

OCMExplore

2.1 Problem

Das Oldenburger Computer-Museum (OCM) bietet seinen Besuchern die Möglichkeit, die Entwicklung der Computertechnik anhand zahlreicher historischer Systeme nachzuvollziehen. Viele der ausgestellten Computer können dabei direkt bedient werden und vermitteln so einen praxisnahen Eindruck ihrer Funktionsweise. Einige besonders seltene oder empfindliche Exponate befinden sich jedoch ausschließlich in Vitrinen und können nicht interaktiv erkundet werden. Zu diesen Exponaten gehört auch der KIM-1, ein Einplatinencomputer aus dem Jahr 1976. Obwohl er als wichtiger Meilenstein der Mikrocomputerentwicklung gilt, können Besucher ihn lediglich betrachten. Informationen über die einzelnen Hardwarekomponenten und deren historische Bedeutung lassen sich nur eingeschränkt vermitteln, wodurch das Potenzial des Exponats nicht vollständig ausgeschöpft wird.

2.2 Idee

Um den KIM-1 auch ohne direkten Zugriff interaktiv erlebbar zu machen, entstand die Idee einer Augmented-Reality-Anwendung. Statt das Originalexponat zu ersetzen, soll dieses durch digitale Inhalte ergänzt werden. Besucher richten dabei ihr mobiles Endgerät auf den KIM-1 und erhalten zusätzliche Informationen direkt am Exponat. Mithilfe interaktiver Infoboxen können einzelne Hardwarekomponenten ausgewählt werden, um weiterführende Informationen zu ihrer Funktion und historischen Einordnung zu erhalten. Ergänzend sollen beispielhafte Programme den grundsätzlichen Ablauf einer Programmausführung auf dem KIM-1 veranschaulichen.

2.3 Projekt

Im Rahmen dieses Projekts wurde eine mobile Augmented-Reality-Anwendung entwickelt, die den ausgestellten KIM-1 digital erweitert. Nach dem Erfassen des Exponats werden virtuelle Inhalte an den passenden Positionen über dem realen Computer eingeblendet. Hierfür dient ein dreidimensionales Modell des KIM-1 als räumliche Referenz, um Infoboxen und Animationen präzise auf den einzelnen Hardwarekomponenten zu positionieren.

Die Anwendung umfasst interaktive Infoboxen mit Informationen zu den wichtigsten Bauteilen des KIM-1 sowie zwei animierte Beispielprogramme. Ein Zählerprogramm und die Berechnung der Fibonacci-Folge werden Schritt für Schritt dargestellt. Dadurch erhalten Besucher einen anschaulichen Einblick in den Aufbau des Computers und die grundlegende Arbeitsweise historischer Mikrocomputersysteme.

2.4 Planung

Nach der Recherche wurden die Anforderungen an die Anwendung gemeinsam festgelegt und die Aufgaben innerhalb der Gruppe verteilt. Dabei orientierte sich die Aufgabenteilung an den Interessen und Vorkenntnissen der einzelnen Gruppenmitglieder. So konnten verschiedene Bereiche, wie die Erstellung des 3D-Modells, die Entwicklung der AR-Anwendung, die Umsetzung der Infoboxen sowie die Programmanimationen, parallel bearbeitet werden. Während des gesamten Projekts fanden regelmäßige Teamabstimmungen statt. Dadurch konnten Fortschritte besprochen, Probleme frühzeitig erkannt und die einzelnen Arbeitsschritte aufeinander abgestimmt werden. Für die gemeinsame Entwicklung und Versionsverwaltung wurde GitLab verwendet, wodurch Änderungen zentral verwaltet und unkompliziert zusammengeführt werden konnten.

2.5 Aufgabenteilung

Name	Aufgaben	Spezialisierung
Luca	Research Gestaltung der Programme Arbeit am Emulator	KIM-1 Experte
Sebastian	Modellieren vom KIM-1 Grundlagen für Animationen am KIM-1 Einrichten vom AR	3D Artist AR
Louisa	Projektmanagement Einrichten Unity Technische Implementation Unity Einrichtung vom AR Animieren von Programmen	Unity AR
Fabian	Research Formulierung von Texten Implementierung von Infoboxen	KIM-1 Experte
Lawend	Research Animieren von Programmen Prototypisieren in Unity	Unity

2.6 Herausforderungen

Die größte Herausforderung im Projekt bestand in der ursprünglich geplanten Umsetzung als webbasierte Augmented-Reality-Anwendung. Ziel war es zunächst, dass Besucher die Anwendung ohne Installation über einen QR-Code direkt im Browser nutzen können. Im Verlauf der Entwicklung stellte sich jedoch heraus, dass diese Umsetzung mit den verwendeten Technologien nicht realisierbar war. Aus diesem Grund musste das Konzept angepasst und die Anwendung als native AR-App entwickelt werden. Dieser Wechsel erforderte zusätzliche Anpassungen in der Planung und Umsetzung, ermöglichte letztendlich jedoch eine stabile und funktionierende Anwendung.

2.7 Fazit

Insgesamt konnte das Projekt erfolgreich abgeschlossen werden. Die Zusammenarbeit im Team verlief gut und durch die Aufteilung der Aufgaben konnten die einzelnen Projektbestandteile parallel entwickelt und anschließend zu einer gemeinsamen Anwendung zusammengeführt werden. Besonders positiv war der hohe Praxisbezug

des Projekts. Anstatt ausschließlich an einer theoretischen Aufgabenstellung zu arbeiten, entstand eine Anwendung für einen realen Einsatz im Oldenburger Computer-Museum. Dadurch war jederzeit ein konkretes Ziel vorhanden, was die Arbeit zusätzlich motivierend machte. Gleichzeitig bot das Projekt genügend Freiraum, eigene Ideen einzubringen und verschiedene Lösungsansätze auszuprobieren. Schlussendlich lässt sich sagen, dass es ein erfolgreiches Projekt für alle Beteiligten war und es mit Zufriedenheit abgeschlossen wurde.

Kapitel 3

Schätze der Vergangenheit

Projektmitglieder: Vanessa Röper, Lennard Sturmman, Silvan Meyerdierk

3.1 Einführung

Das Modul "Bachelorprojekt XR Museum" fand dieses Semester in Kooperation mit dem Computermuseum OCM in Oldenburg statt. Das Museum zeigt die Entwicklung von Computern von ihren Anfängen bis zur heutigen Zeit und ermöglicht dabei einen Einblick in die Technik, die dahintersteckt.

Bei unserem ersten Besuch im Museum fand eine kleine Führung statt, bei der die Grundidee zur Entstehung des Museums sowie die gesamte Ausstellung vorgestellt wurden.

Der Museumsleiter erzählte davon, dass er die Erfahrung gemacht hat, dass viele Besucher die Informationstafeln zu den Ausstellungsstücken nicht lesen würden, da dies zu umständlich sei. Außerdem könnte es für viele Besucher interessant sein, sich die Hardware der Computer selbst genauer anzugucken oder auszuprobieren, was außerhalb der Führung momentan nicht möglich ist. Das hat zur Folge, dass der Lerneffekt für die Besucher verloren gehen kann, da sie meist nur erahnen können, wie das Gerät genau funktioniert.

Des Weiteren haben wir festgestellt, dass Besucher ohne eine geschulte Führung der Museumsleiter eher ziellos durch das Museum streifen und an potenziell interessanten Ausstellungsstücken vorbeigehen.

Aus diesen Gründen fiel die Wahl auf die Entwicklung einer Smartphone-App, mit der

die Besucher eine Art Schnitzeljagd durch das Museum machen können. Dabei sollen mehrere kleine Rätsel gelöst werden, um die Besucher dazu zu bringen, gezielt mit den einzelnen Ausstellungsstücken zu interagieren. Die App soll es ermöglichen, die Funktionsweise und den Aufbau der Computer eigenständig zu entdecken. Sie soll eine Ergänzung zu der angebotenen Führung sein, bei der die Besucher im eigenen Tempo das Museum erkunden können.

3.2 Konzept

Im Laufe der Schnitzeljagd soll der Besucher die Ausstellungsstücke mit Hilfe von vier verschiedenen Minispielen kennenlernen. Jedes Spiel konzentriert sich auf einen anderen Computer im Museum. Die Schnitzeljagd soll dabei von einem Maskottchen moderiert werden. Er trägt den Namen "C π Rat" und ist ein Pirat ausgestattet mit einem Bildschirm als Gesicht, Holzbein, Piratenhut, Augenklappe und einer Enter-Taste als Haken (siehe Abb. 1).

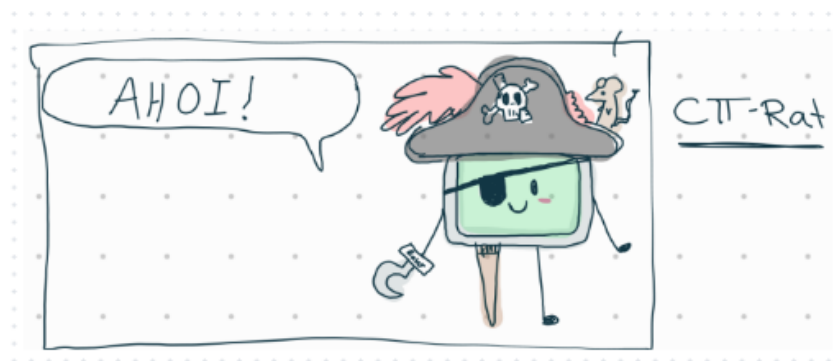


Abbildung 3.1: Maskottchen C π Rat Skizze

Das Spiel beginnt bei dem ersten Computer, den man im Museum sieht, den PDP-8/M, der als großer Kasten hinter einem Absperrband steht. Während der Museumsführung wird gezeigt, wie man diesen Computer bedient und einfache Eingaben für Berechnungen vornehmen kann. Allerdings haben die Besucher nicht die Möglichkeit, selbst diesen Computer auszuprobieren. Im ersten Spiel sollte genau das ermöglicht werden, indem die Kippschalter-Eingaben und Ausgaben in Form von roten LEDs mit AR auf den PDP-8/M projiziert werden. Dadurch können die Besucher das Gerät über ihr Smartphone alleine bedienen und verstehen, wie es funktioniert (vgl. Abb. 2).

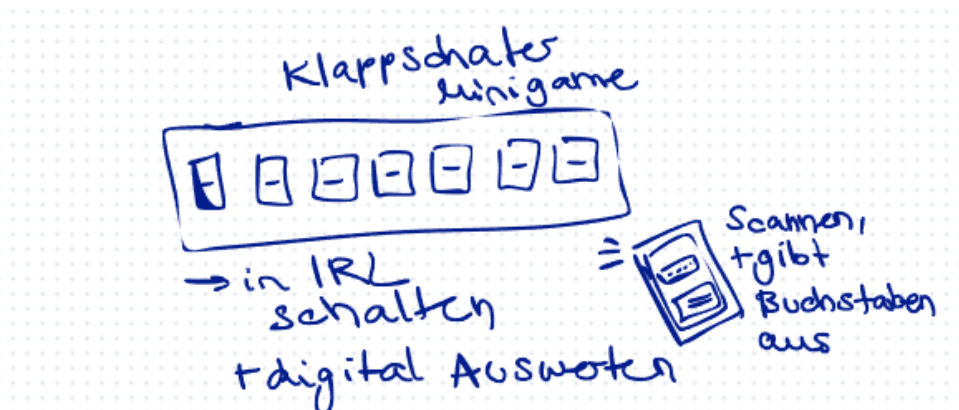


Abbildung 3.2: Kippschalter Minispiel Skizze

Das zweite Minispiel sollte sich mit dem Problem befassen, dass die Hardware der einzelnen Computer nur während der Museumsführung für Besucher sichtbar ist. Die App soll es ermöglichen, dass die Besucher die Komponenten auch eigenständig entdecken und sich darüber informieren können. Das Ziel des Spiels ist es, einen Apple II Computer zusammenzubauen. Die einzelnen Komponenten sind dabei auf dem Smartphone Bildschirm beweglich und können per Drag&Drop vom Spieler verschoben werden. Als Referenz dient dabei ein fertig zusammengebauter Apple II Computer, den sich die Besucher anschauen können. Zu jeder Komponente soll es ein 3D-Modell und einen Erklärtext geben, damit der Besucher auch verstehen kann, was die einzelnen Funktionen sind (vgl. Abb. 3).

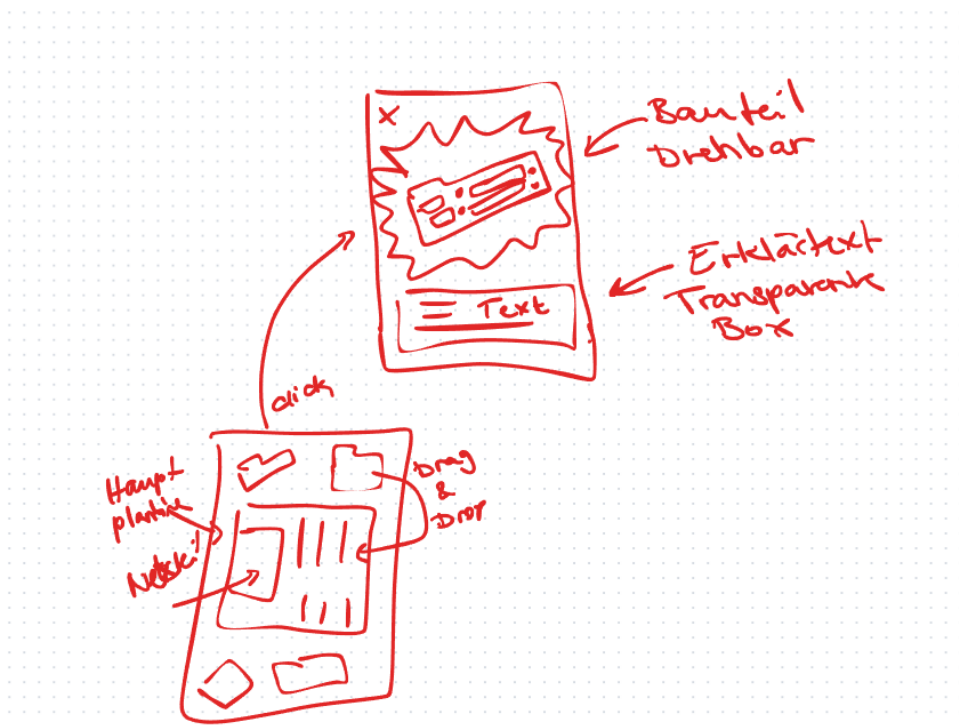


Abbildung 3.3: PC Zusammenbau Minispiel Skizze

Mit dem dritten Minispiel sollen die Besucher dazu angeregt werden, sich etwas genauer mit den einzelnen Computern zu beschäftigen und auch die Infotafeln dazu zu lesen. Im Zuge eines Such-Spiels werden den Besuchern mehrere unkenntlich gemachte Computer aus dem Museum gezeigt. Die Besucher müssen dann mit Hilfe von vier möglichen Antworten sowie ihrer Umgebung herausfinden, um welches Gerät es sich handelt. Bei falschen Antworten soll es Hinweise geben und wenn die Spieler richtig getippt haben, soll es einen kleinen Erklärtext zu dem Computer geben.

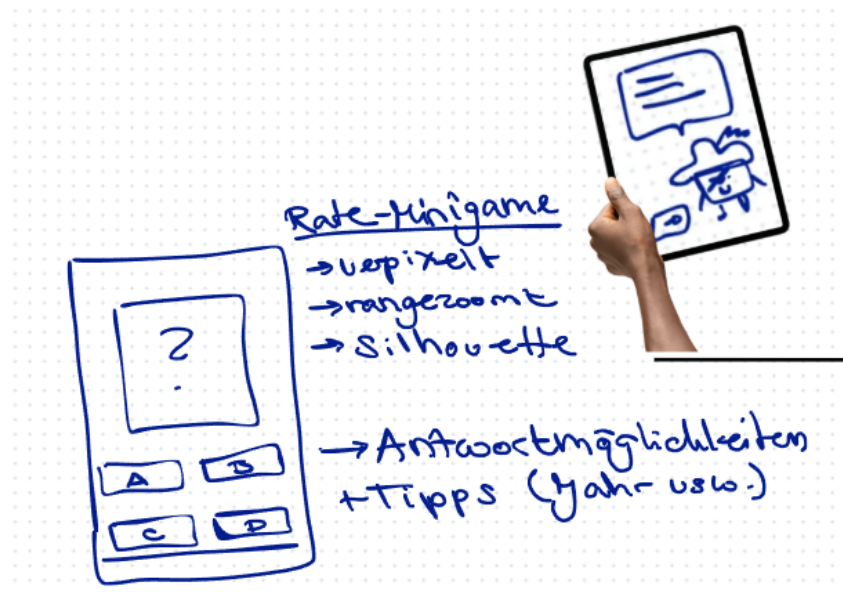


Abbildung 3.4: Quiz Minispiel Skizze

Das letzte Spiel der Schnitzeljagd beschäftigt sich mit der Programmiersprache BASIC, welche auf den älteren Computern im Museum verwendet wird. Die Besucher haben im Museum die Möglichkeit, eigenständig kleine Programme an den Ausstellungs-Computern zu schreiben. Dieser Prozess vom genauen Abtippen der Befehle auf einem ungewohnten Interface kann für einige Besucher abschreckend oder frustrierend wirken, weshalb die Programmierung abgebrochen werden kann. Mit dem letzten Spiel soll es vereinfacht werden, einfache Programme zu erstellen. Mit Hilfe von Code-Blöcken und Lückentexten soll die Programmierung vereinfacht und verschleunert werden, sodass die Besucher auch die Ergebnisse der Programme sehen können (siehe Abb. 5).

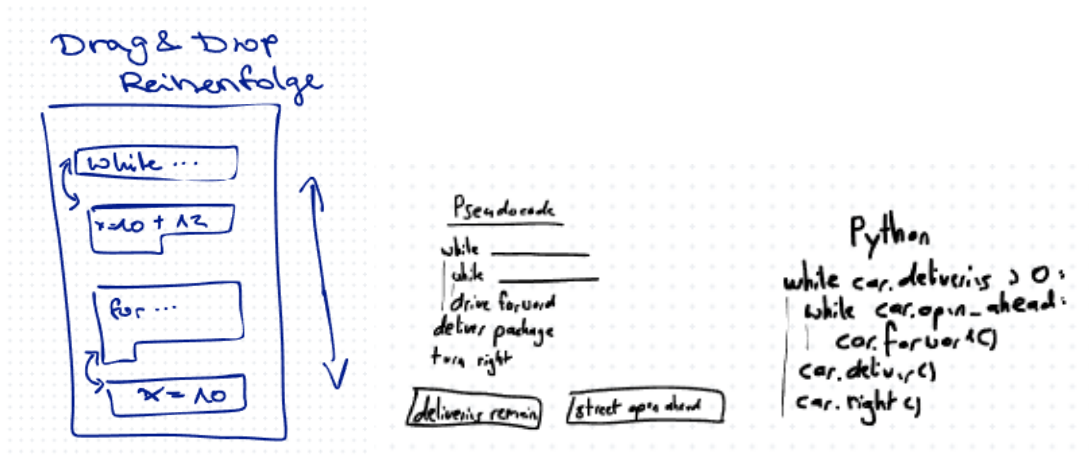


Abbildung 3.5: Code Minispiel Skizze

Die Spiele sollen einen roten Faden durch das Museum von Start bis Ende darstellen. Im Eingangsbereich des Museums wird ein Aufsteller platziert, der die Besucher über diese App informiert und einen Download Link bereitstellt. Alternativ kann das Museum Leihgeräte zur Verfügung stellen, auf denen die App vorinstalliert ist. Wenn die Besucher die Schnitzeljagd erfolgreich beendet haben, können sie sich einen Preis am Tresen im Eingangsbereich abholen. Der Ablauf der Schnitzeljagd ist in Abbildung 6 noch einmal aufgeführt. Mit diesem Schaubild wurde die Spielidee dem Museum vorgestellt.

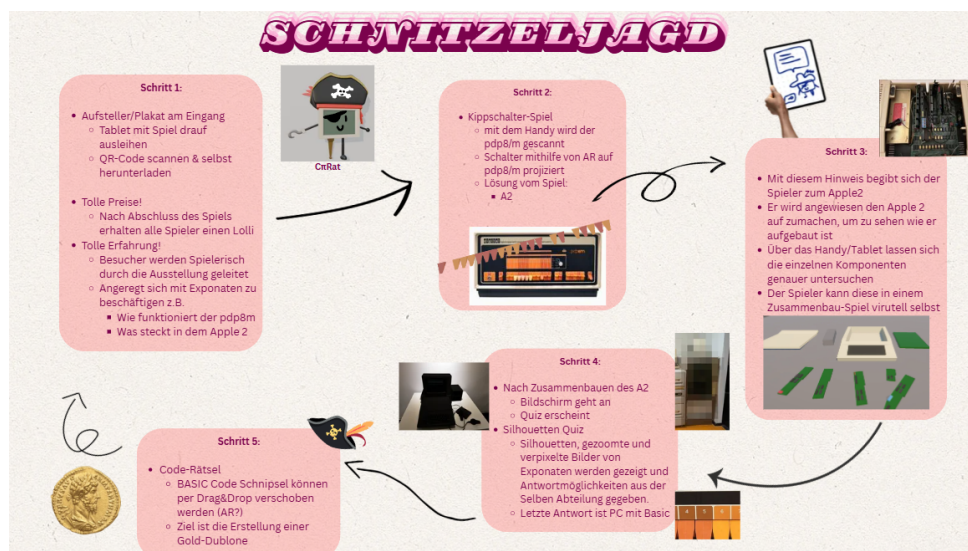


Abbildung 3.6: Schnitzeljagd Plan

3.3 Anforderungen

Um sowohl eine strukturierte Entwicklung als auch einen erfolgreichen anschließenden Einsatz der Anwendung im Museum zu gewährleisten, müssen im Vorfeld definier-

te technische und inhaltliche Anforderungen erfüllt sein. Diese Kriterien lassen sich grundlegend in zwei Perspektiven unterteilen: Einerseits die Entwicklungsanforderungen, die während der Entwicklung relevant sind, sowie die Systemanforderungen, die gelten, um das finale Produkt zu verwenden.

Die technische Umsetzung der Anwendung basiert auf der Game Engine Unity in der Version 6000.3.8f1. Um die Augmented-Reality-Funktionen auf den mobilen Endgeräten zu realisieren, ist die Integration des Unity AR Foundation Package zusammen mit dem neuen Unity Input System zwingend erforderlich. Die Erstellung und Texturierung der dreidimensionalen Objekte erfolgt in der Open-Source-Software Blender, während zweidimensionale Grafiken und Texturen mit Adobe Photoshop bearbeitet werden.

Für unsere verschiedenen Minispiele werden eine Vielzahl von Assets benötigt. Ein zentrales Element ist das dreidimensionale Modell des Spielleiters und Maskottchens namens "C π Rat", für das verschiedene Posen und Gesichtsausdrücke modelliert werden müssen, um eine dynamische und emotionale Nutzererfahrung zu ermöglichen. Unser erstes Spiel erfordert einzeln modellierte Kippschalter und Kontrolllampen, die in ihrer visuellen Anordnung exakt dem historischen Vorbild des PDP-8/M entsprechen. Das Zusammenbauspiel benötigt ein detailliertes, modular aufgebautes 3D-Modell eines Apple II-Computers, dessen einzelne Komponenten separat greifbar und beweglich sein müssen. Für das Quiz-Spiel müssen Fotos von Museumsexponaten in Photoshop zu kontrastreichen Silhouetten aufbereitet werden. Für alle Spiele übergreifend werden ebenfalls Elemente der Benutzeroberfläche wie Icons und Sprechblasen benötigt, welche auch in Photoshop erstellt wurden.

Neben den rein technischen und visuellen Komponenten müssen im Vorfeld der Entwicklung auch inhaltliche Anforderungen durch fachliche Recherche abgedeckt werden. Einerseits betrifft dies das technische Verständnis der zu simulierenden historischen Geräte, sowie andererseits den realen physischen Aufbau des Apple II zu analysieren und für die Spiellogik aufzubereiten. Weiter erfordert das Quiz-Spiel eine Liste mit korrekten Antworten und plausiblen falschen Antwortmöglichkeiten.

Zur Nutzung des finalen Prototypen wird ein AR-fähiges mobiles Endgerät mit dem Android Betriebssystem benötigt.

3.4 Entwicklungsprozess

Nach der Ideenfindung wurden regelmäßige Meetings, zusätzlich zu den Stand-up-Runden freitags, gehalten und Aufgaben wurden verteilt. Begonnen wurde die Umsetzung mit den reinen Funktionen in Unity und vorerst Platzhalter für die 3D Objekte, welche direkt am PC getestet werden konnten. Diese wurden so gebaut, dass sie einfach für die Anwendung oder auch den AR Bereich eingesetzt werden können.

Anfangs standen die Grundfunktionen im Fokus. Bei den Kippschaltern beispielsweise die Funktion diese umzulegen und eine Bitfolge von diesen auszulesen. Nachdem die Funktionen standen, wurden diese durch Details und Erweiterungen ergänzt. Das Apple II Spiel wurde zum Beispiel so erweitert, dass Infos über die einzelnen Teile angezeigt werden, wenn diese angeklickt werden.

Um das ganze in AR anzuwenden, wurde mit 2D Image Tracking gearbeitet. Da hier nicht jedes Bild geeignet ist, wurde viel Zeit verschwendet mit Bildern zu arbeiten, die nicht funktioniert haben. Das hat dafür gesorgt, dass die Test App nicht funktioniert hat und das 3D Objekt nicht auf die Fläche projiziert wurde. Nach Herumprobieren und Recherchen wurde das Problem dann mit der `arcoreimg.exe` gelöst. Mit dieser kann man Bilder darauf testen, wie gut diese für das Tracking in Unity geeignet sind. Nun konnten Bilder genutzt werden, die auch mit der Funktion kompatibel waren.

Um Feedback zu sammeln, wurde für den zweiten Museumsbesuch ein Prototyp mit 3 unfertigen Level mitgebracht. Hier funktionierte das Projizieren der Schalter auf den PDP 8/M schon gut. Um die Größen der Schalter richtig anzupassen, wurden Maße genommen. Auch wurde dieser Besuch genutzt, um Bilder für Quiz-Inhalte und 3D Modelle zu machen. Bei dem Besuch stellte sich auch heraus, dass die ursprüngliche Idee, den Besucher anzuregen, in den Apple II hineinzuschauen und dort zu vergleichen, nicht im Sinne des Museums Betreiber war. Daraufhin wurde das Apple II Spiel insoweit angepasst, dass der Nutzer mit AR die richtig zusammengebaute Platine auf den Apple II projizieren kann und dies als Vergleich nutzt. Aus diesem Grund wurden auch direkt Maße und Bilder von dem Apple II genommen beziehungsweise gemacht.

Das Wechseln zwischen zwei AR Umgebungen brachte Probleme mit sich. Aus diesem Grund wurde eine AR-Szenen verwendet und mit einem Skript wurde sowohl das 3D Objekt als auch die 2D Fläche, auf welches dieses projiziert werden sollte, zum passenden Zeitpunkt gewechselt.

Nach dem Besuch wurde alles verfeinert und an das Feedback angepasst. Das Basic Level wurde ebenfalls entwickelt. Ein weiterer Schritt war es, die Dialoge von dem C π Rat mit einem einheitlichen Interface in die verschiedenen Level einzubinden. Das letzte Problem, das vorhanden war, ist das Wechseln zwischen der AR-Szene und der Apple II-Szene gewesen. Einerseits durfte die AR-Szene nie ganz gelöscht werden, da dies für Fehler sorgte. Andererseits musste der Fortschritt in der Apple II-Szene gespeichert werden. Gelöst wurde das ganze so, dass beide Szenen, aber auch das Menü gleichzeitig mit Additive Szene geladen werden, aber ein Skript nur die richtige Szene aktiviert und gleichzeitig die anderen deaktiviert. Um das ganze einfacher zu machen, wurden alle Inhalte aus den drei Szenen jeweils in ein übergeordnetes Empty-Objekt gepackt, welches einfach ein- und ausgeschaltet werden kann.

3.5 Prototyp

Im Folgenden wird der aktuelle Stand sowie der Ablauf der einzelnen Spiele erläutert. Dabei wird auf jedes Level einzeln eingegangen.

3.5.1 Kippschalter-Level

Der Spieler muss den PDP-8/M finden und mit der Kamera auf diesen zeigen. Sobald die App den Computer erkennt, werden auf diesen die Schalter an der richtigen Position gepawnt. Danach folgt eine Erklärung vom C π Rat, wofür welche Schalter in dem Level genutzt werden können und was der Hinweis ist, um das Level zu meistern. Und zwar die zwölf Bit-Schalter, so wie der Deploy-Schalter. Nach dem Umschalten des Deploy-Schalters wird die Bit Reihenfolge bestätigt und die zugehörigen LEDs leuchten, um die Eingabe anzuzeigen. Ist die Eingabe falsch, kann einfach eine neue Eingabe erfolgen. Wenn der richtige Code eingegeben wurde, wird mit dem Ergebnis auf das nächste Level hingewiesen.

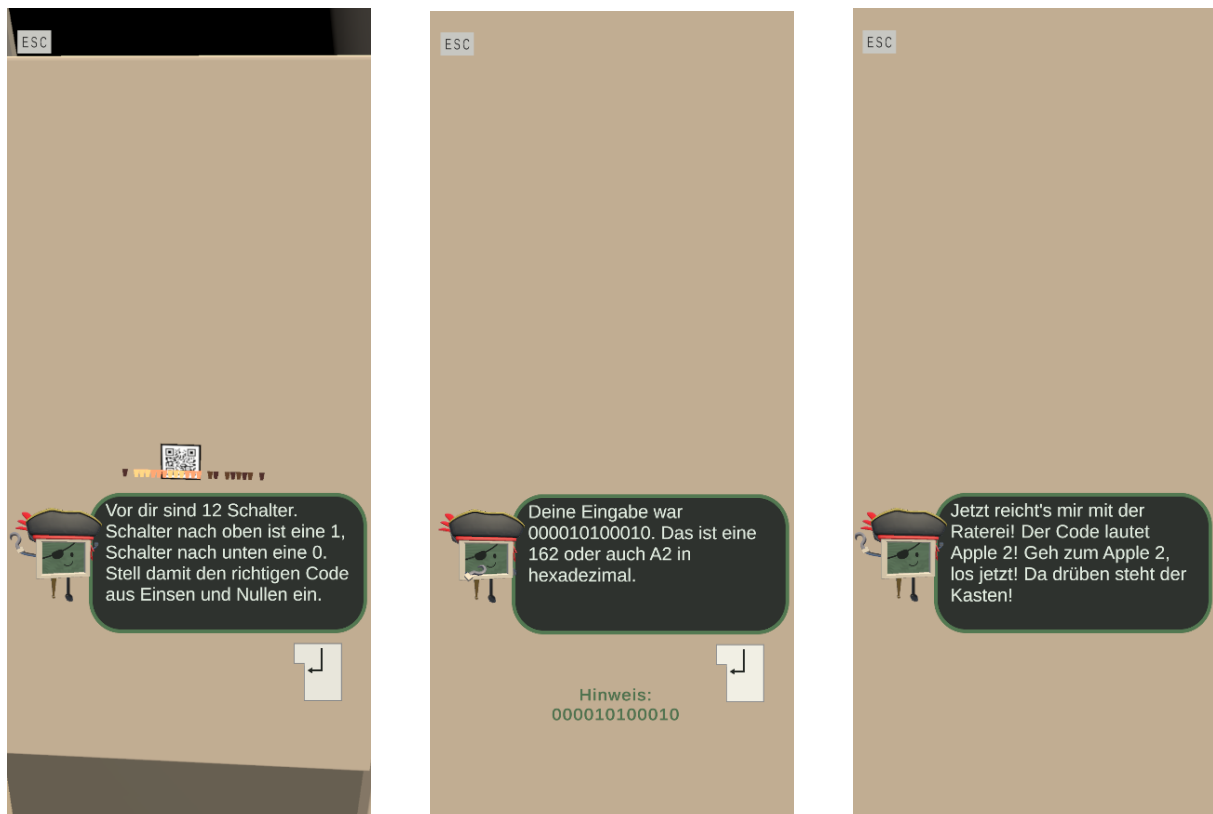


Abbildung 3.7: Kippschalter-Spiel Screenshot

3.5.2 Apple II Zusammenbau-Level

Der Spieler sieht sich nach dem erfolgreichen Abschließen des Kippschalter Levels mit dem Lösungswort “A2” präsentiert, welches inhaltlich als Überleitung zum Apple II-Computer dient, wo das nächste Spiel startet.

Zunächst fordert die Anwendung den Nutzer dazu auf, den physischen Apple II im Museum mit der Kamera seines Endgeräts zu scannen. Mittels Augmented Reality wird daraufhin ein detailliertes Modell des Innenlebens des Apple II über das Exponat projiziert. Über ein spezifisches UI-Element in Form eines Maulschlüssels wird der Nutzer nun in den zweiten Teil des Spiels geleitet.

Dort angekommen, wird er von “ $C\pi$ Rat” begrüßt und darauf hingewiesen, dass ein in der Szene befindlicher Apple II nicht zusammengebaut sei. Mithilfe der vorherigen Szene soll der Spieler diesen nun Stück für Stück zusammenbauen. Während der Montage kann mithilfe von Buttons zwischen der AR-Ansicht des zusammengebauten PCs und der Zusammenbau-Szene gewechselt werden. Innerhalb der Zusammenbau-Szene ist der Spieler in der Lage einzelne Komponenten anzuklicken, und weitere Informationen in Form von Namen und Beschreibung zu erlangen.

Sobald der Montageversuch abgeschlossen ist, greift eine spielerische Feedback-Schleife. Bei einer fehlerhaften Zusammensetzung initiiert das System eine visuelle Fehlermel-

dung in Form von virtuellen Rauch, der aus dem PC-Gehäuse aufsteigt. Das Maskottchen weist über eine Sprechblase auf den Fehler hin, woraufhin das Gehäuse für Korrekturen wieder geöffnet werden muss. Der Rauch dient hierbei als visueller Indikator, um die fehlerhaft platzierten Komponenten zu lokalisieren und einen gezielten Austausch zu ermöglichen.

Wurde der Computer hingegen fehlerfrei zusammengebaut, erhält der Nutzer eine positive Validierung durch den Spielbegleiter. Gleichzeitig schaltet sich der Monitor des Computers an. Im Bildschirm zu sehen ist das nächste Spiel der Schnitzeljagd. Durch anklicken des Monitors wird ein Übergang in das nächste Spiel initiiert.

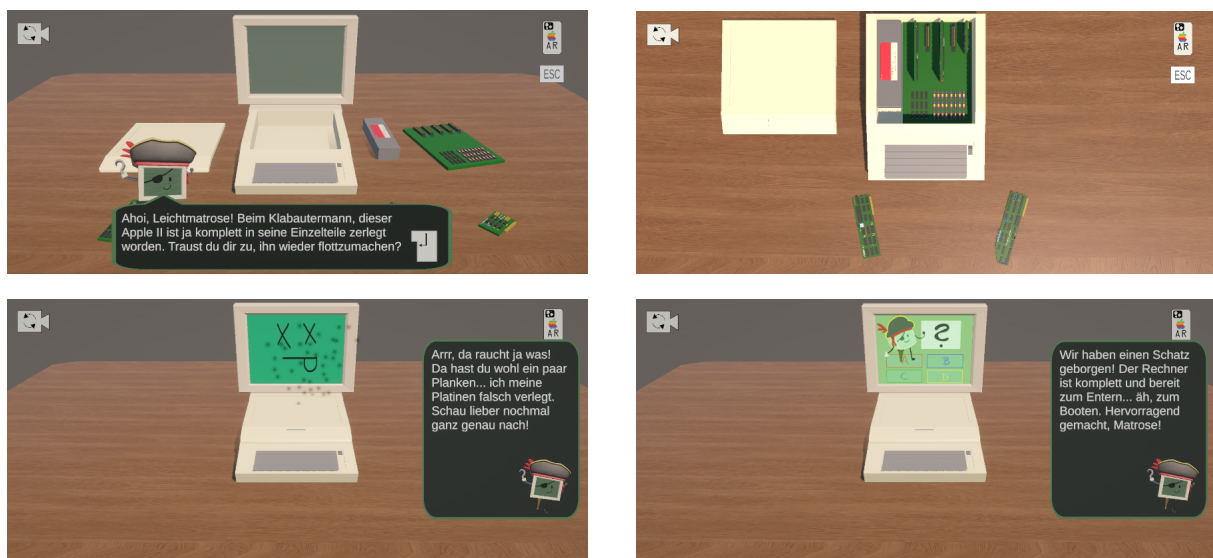


Abbildung 3.8: PC Zusammenbau-Spiel Screenshot

3.5.3 Quiz-Level

Der Spieler sieht anfangs eine schwarze Silhouette eines Objektes, welches sich irgendwo im Museum befindet. Darunter sind vier Antwortmöglichkeiten, die dem gesuchten Objekt ähnlich sind oder sich in der Nähe befinden. Der Spieler hat nun die Aufgabe, sich im Museum umzusehen und anhand der Silhouette das Objekt zu identifizieren. Dafür kann er die Infotafeln, die im Museum hängen, zur Hilfe nehmen.

Wenn der Spieler falsch antwortet, wird das Bilderrätsel etwas einfacher. Zuerst wird ein Hintergrund in das Bild eingefügt, wodurch Hinweise auf den Standort gegeben werden sollen. Danach wird das Objekt bei jeder falschen Antwort ein Stück sichtbarer gemacht, bis das ganze Objekt in Farbe dargestellt wird.

Wenn der Spieler richtig geraten hat, wird ein kleiner Infotext mit Fun-Facts zu dem jeweiligen Objekt angezeigt und anschließend die nächste Silhouette gezeigt. So wird der Spieler durch die verschiedenen Abteilungen geschickt, wobei immer eine Silhou-

ette pro Abteilung angezeigt wird. Wenn alle Bilder-Rätsel gelöst wurden, geht es zum nächsten Spiel.

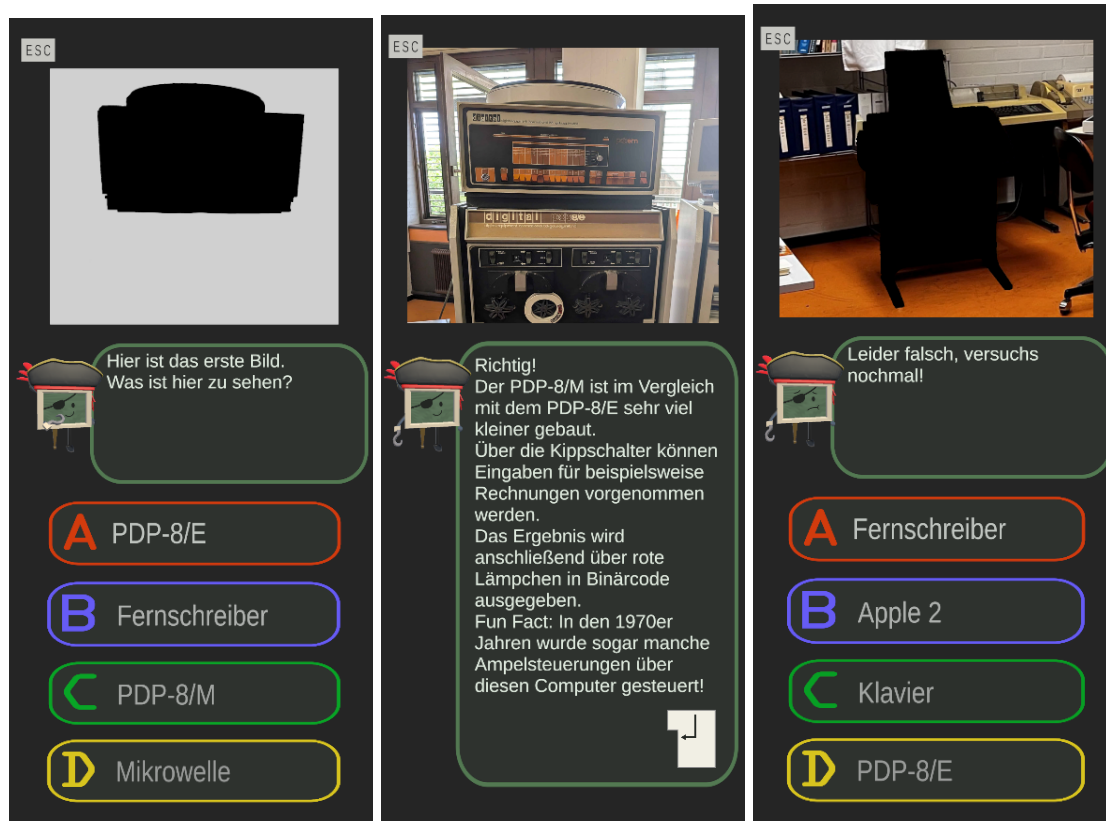


Abbildung 3.9: Quiz-Spiel Screenshot

3.5.4 Code-Level

Im letzten Spiel müssen Code-Blöcke mit Basic Code in die richtige Reihenfolge gebracht werden. Auf dem oberen Teil des Bildschirms ist eine Anzeige, die das Ergebnis des Codes Schritt für Schritt abbildet. Darunter gibt es vier Codeblöcke, die in die richtige Reihenfolge gebracht werden müssen. Mit dem RUN-Button kann der Spieler den Code ausführen und sich das Ergebnis anschauen. Bei dem Basic Code ist es wichtig, in welcher Reihenfolge die Schritte ausgeführt werden, da zuerst ausgeführter Code von dem danach überdeckt wird.

Wenn der Spieler den Code so angeordnet hat, sodass nichts voneinander verdeckt wird, hat er das Rätsel gelöst und es ist eine Goldmünze mit einer Enter-Taste darin zu sehen.

Anschließend gibt es einen kleinen Abschlusstext vom $C\pi$ Rat, indem er erneut auf die Computer hinweist, an denen man selbst Basic-Programme eingeben kann. Des Weiteren wird erzählt, dass man sich nach erfolgreichem Abschluss der Schnitzeljagd einen Preis in Form eines Lollis am Anfangstresen abholen kann.

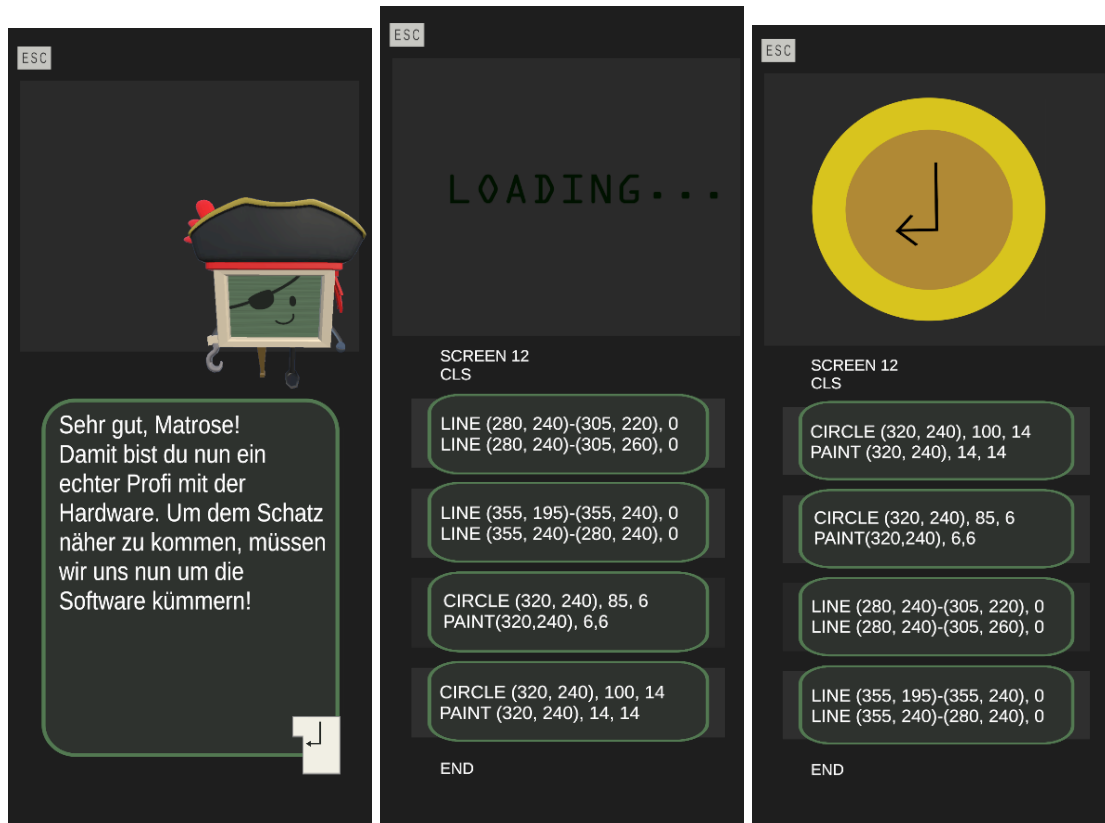


Abbildung 3.10: Code-Spiel Screenshot

3.5.5 Technische Dokumentation

Für jedes Level wurde eine eigene Scene im Unity Projekt angelegt. Jedes Spiel besitzt eigene Skripte und Besonderheiten, die im Folgenden erläutert werden.

AR Scene: "Kippschalter"

Hier findet alles mit AR statt. Hierfür befinden sich alle wichtigen Dinge in ARRoot.

Einmal die UI elemente, also der C π Rat und das Canvas mit zugehörigen Manager. Auf dem Manager sind die Skripte "Intro Manger" und "Apple II Intro Manger", welche für die Dialoge in den AR Szenen zum Kippschalter und Apple II zuständig sind. Diese reagieren auf das Spawnen von dem Kippschalter so wie der Apple II Platine. Auf den Prefabs sind jeweils ein Script, welches beim Starten zu dem richtigen Schritt in dem Manager springt. Auf der Platine ist das "Platine" und auf den Kippschalter das "Schalter Zustand" Script, welches neben den Funktionen der Schalter auch hierfür zuständig ist.

Auch die XR Komponenten sind hier zu finden. Die XR Origin und die AR Session. In dem XR Origin wird die Reference Image Library eingestellt, so wie das 3D-Objekt

welches gespawnt werden soll, sobald das Image aus der Library erkannt wurde. Auf dem XR Origin befindet sich ebenfalls ein Script, das "Change Image Lib" heißt und die Funktion hat, die Library und das 3D-Modell zu wechseln. Auch das Camera Offset mit der Main Camera befindet sich in dem XR Origin. Auf dem Camera Offset befindet sich das "AR Touch Manager" Script, welches dafür sorgt, dass die Eingaben am Handy richtig verarbeitet werden und die Schalter umgeschaltet werden können.

Kippschalter Funktion

Die Kippschalter sind so aufgebaut, dass jeweils alle 3D-Schalter in einem Parent, dem Kippunkt, sind, um sie zum Umschalten zu rotieren. Jeder Kippunkt besitzt ein Script, das "Schalter Einzel" Script, welches für den Zustand des einzelnen Schalters zuständig ist. Hier werden Rotation, Material und der Schaltzustand gespeichert und verändert.

Dieses Script kommuniziert mit dem Script "SchalterZustand", welches für alle Schalter den aktuellen Binär-, Dezimal- und Hexawert berechnet sowie die Kommunikation mit dem Intro Manager zur Übertragung der Eingaben übernimmt. Auch die LEDs werden hier, je nach Eingabe, angesteuert. Dieses Script sitzt auf dem Prefab der gesamten Schalter. Die LEDs sind ebenfalls in einem Kippunkt, welches das "LED_Behavior" Script hat. Dieses sorgt für das An- und Ausschalten der LEDs.

AR Tracking

In dem Ordner "ImageTrackingResources" befinden sich die Image Reference Libraries sowie ein Ordner mit den dafür genutzten Bildern.

In den Libraries kann man Bilder hinzufügen, dort die physische Größe angeben und die Library erkennt die 2D-Fläche.

Um ein passendes Bild zu finden ist in dem Unity Projektordner unter \Library\PackageCache\com.unity.xr.arcore@a8637e6ab6bf\Tools~\Windows eine arcoring.exe, welche man im cmd im beschriebenen Pfad mit dem Befehl: "arcoring.exe eval-img -input_image_path=dog.png" ausführen kann. Dort wird für das angegebene Bild ein Wert von 0-100 zurückgegeben. Je größer desto besser für das Image Tracking.

Um eine neue Image Tracking Library zu verwenden, muss diese in dem XR Origin Objekt in der AR-Szene eingestellt werden.

Umschalten zwischen Menu, AR und Building

Um zwei Probleme zu beheben, wird zwischen dem Menü, der AR-Szene und der Apple II-Szene mit Additiven Laden und Verstecken der anderen Szenen gearbeitet.

Durch diese Methode wird der Spielstand im Apple II Level gespeichert, auch wenn man zwischen der AR- und der Apple II-Szene wechselt. Auch wird gewährleistet, dass die AR-Umgebung nur einmal existiert und nicht gelöscht wird, solange diese benötigt wird und somit keine Probleme entstehen. Das Script "Scene Switch Controller" befindet sich in der Menu-Szene auf dem SSceneSwitchArBauObjekt. Hier müssen die Indexe der AR- und Apple II-Szene eingegeben werden. Zu finden sind diese unter File->Built Profiles -> Scene List. Dort sind die Indexe rechts neben den jeweiligen Szenen zu sehen. Damit dies funktioniert, müssen alle Inhalte der jeweiligen Szenen in einem Objekt mit dem Tag "MenuRoot", "ARRoot" und "BauRoot" sein.

Spiel 2: PC Building Simulator

Das zweite Spiel verfügt über eine AR-Szene zum Betrachten des Inneren eines Apple II und eine Zusammenbau-Szene in einer vollen virtuellen Umgebung anknüpfend dazu.

Szenenwechsel und UI-Management

Der Wechsel aus der initialen AR-Umgebung in die Szene "PcBuildingSimulator" erfolgt durch das Schraubenschlüssel UI-Element. Die generelle Szenen-Navigation, wie etwa der Rücksprung in die AR-Ansicht über das AR-Handy-Icon oder der Wechsel ins Hauptmenü, wird durch einen zentralen "MenuManager" gesteuert. Während der Bauphase ermöglicht das Skript "CameraSwitch" dem Spieler zudem die Möglichkeit, zwischen Front-View-Kamera und Top-Down-Kamera zu wechseln.

IntroManagerPC

Sobald die Zusammenbau-Szene geladen ist, greift das Skript "IntroManagerPC", welches an das GameObject "GameManager" gebunden ist. Das Skript steuert die Begrüßungs-Dialoge des Maskottchens ("CπRat"). Die angezeigten Texte können über Arrays im Inspektor definiert werden. Nach dem Durchklicken aller Intro-Texte wird das dazugehörige introPanel deaktiviert und die Variable "isIntroActive" auf false gesetzt, wodurch die Interaktion mit den Bauteilen freigegeben wird.

Bauteil-Logik

Die interaktiven PC-Komponenten in der Szene sind mit den Skripten "DraggableComponent" und "ComponentInfo" ausgestattet.

Das Skript "DraggableComponent" nutzt das neue Unity Input System und verwaltet die Drag-und-Drop-Mechanik. Um visuelles Clipping zu vermeiden, passt das Skript

die Höhe der Bauteile automatisch an, sobald die Top-Down-Ansicht aktiv ist. Zusätzlich kommuniziert das Skript vor dem Loslassen mit dem GameManager, um Einbauregeln abzufragen (beispielsweise muss die Hauptplatine zwingend vor den kleineren Platinen platziert werden). Darüber hinaus ist das Skript mit Sicherheitsabfragen versehen, wodurch keine Interaktionen mit den 3D-Objekten möglich sind, während UI-Einblendungen aktiv sind. Zudem hält das Skript Referenz auf einen lokalen Rauch-Effekt. Dieser wird vom GameManager ausgelöst, sobald der Computer fehlerhaft zusammengebaut wurde, und automatisch wieder deaktiviert, sobald der Spieler das entsprechende Bauteil erneut anfasst.

InspectManager

Wird ein Bauteil in der Szene nur kurz angeklickt (weniger als 0,25 Sekunden), wird die Logik an den InspectManager übergeben, welcher an den GameManager gekoppelt ist. Das Skript ermittelt zunächst die aktiv verwendete Kamera (Front oder Top-Down) und wählt den entsprechenden 3D-Ankerpunkt vor der Linse. Über die Coroutine "AnimateToCamera" wird das Objekt fließend an diesen Punkt animiert, wobei die im "ComponentInfo"-Skript definierten, kameraspezifischen Offsets angewendet werden. Während der Betrachtung öffnet sich das zugehörige UI-Panel mit Titel und Beschreibung des jeweils angeklickten Bauteils. Um Clipping-Fehler oder ungewollte Eingaben zu vermeiden, blockiert das "InspectManager"-Skript während der Betrachtung alle weiteren Interaktionen mit anderen Bauteilen sowie die UI-Buttons für den Kamera- und Szenenwechsel.

Einbauzonen

Die Zielorte für die Komponenten sind als GameObjects mit dem Namen "Trigger" in der Szene platziert und besitzen das angebundene Skript "DropZone". Diese Zonen steuern das visuelle Highlighting bei Annäherung eines Bauteils und arbeiten eng mit dem Skript "DraggableComponent" zusammen. Das Skript prüft, ob die herangezogene Komponente der erwarteten ID entspricht (für die Korrektheitsprüfung). Basierend auf diesem Ergebnis steuert die Coroutine "AnimatePlacement" die interpolierte Einrast-Animation. Ist das Bauteil inkorrekt, wird es mit einem leichten Rotations-Offset platziert, um den Fehler visuell zu verdeutlichen. Wird ein Bauteil wieder ausgebaut, stoppt das DropZone-Skript umgehend den Rauch-Effekt (wenn aktiv), teleportiert das Objekt auf seine Startposition und blendet im Falle des Gehäusedeckels das UI-Feedback wieder aus.

Nachträgliche Bauteil-Interaktion

Um dem Spieler Fehlerkorrekturen zu ermöglichen, sind die final eingerasteten Komponenten mit dem Skript “PlacedPartInteraction” versehen. Wird ein bereits platziertes Bauteil via Raycast angeklickt, prüft dieses Skript zunächst mithilfe des “GameManager”-Skriptes, ob die Ausbau-Vorbedingungen erfüllt sind (z.B. alle kleinen Platinen müssen vor der Hauptplatine entfernt werden). Ist der Ausbau möglich (CanRemove = true), wird die entsprechende DropZone angewiesen, die dort verbaute Komponente wieder in den Ausgangszustand zu versetzen. Dabei wird ebenfalls der eingebaute globale Klick-Cooldown respektiert, der implementiert wurde, um ungewollte Doppeleingaben zu verhindern.

GameManager & MonitorInteraction

Der GameManager prüft im Hintergrund fortlaufend den Zustand des Computers. Sobald der Gehäusedeckel geschlossen wird, iteriert das Skript über eine Liste aller in der Szene befindlichen DropZones (inklusive inaktiver Objekte). Bei fehlerhaft verbauten Teilen wird ein zufälliger Fehler-Text aus dem Array ausgewählt und mithilfe des C π Rat-UI ausgegeben.

Das Skript “MonitorInteraction”, welches auf dem Monitor GameObject liegt, ist für das visuelle Feedback auf dem Bildschirm zuständig. Im Fehlerfall wird das Material des Bildschirms zu einem Smiley. Das Skript nutzt dabei die interne Status-Variable “isSuccessMode”, um Raycast-Klicks des Spielers zu blockieren, solange der PC fehlerhaft zusammengebaut ist. Bei korrektem Zusammenbau wird das Material zu einem Einführungsbild des nächsten Levels. Der Monitor wird dann zu einem anklickbaren Trigger, um den Szenenwechsel in das Folgelevel einzuleiten.

Spiel 3: v1 (Quiz-Spiel)

Die Logik des Quiz-Spiels ist hauptsächlich mit dem Canvas verbunden. Hier befinden sich alle Elemente, die auf dem Smartphone-Display angezeigt werden. Im Canvas befindet sich ein QuizPanel, welches das einfache Aktivieren bzw. Deaktivieren der gesamten Oberfläche ermöglicht. Für die Ansicht mit der Frage und den Antwortmöglichkeiten gibt es das QuestionPanel, welches aus den einzelnen Buttons sowie einem Image für die gesuchte Silhouette und einem kleinen Hinweistext besteht. Für den Informationstext, der angezeigt wird, wenn eine Frage richtig beantwortet wurde, gibt es das InfoPanel, welches nur aus einem großen Textfeld besteht. Des Weiteren gibt es einzelne Gameobjekte, die das Einblenden des Weiter-Buttons und des Zurück-Buttons ermöglichen.

QuizManager

Damit der Ablauf des Quiz-Spiels funktioniert, gibt es in der Manager-Komponente das Gameobject "QuizManager". Diese erhält das Script "Quiz Manager". In dieser Komponente sind die einzelnen Fragen und Antworten gespeichert, die im Quiz angezeigt werden. Dafür wird ein Array aus Quiz-Objekten verwendet, die in dem Script "Quiz" definiert sind. Die Klasse Quiz besteht aus den einzelnen Textfeldern für die Einleitung, die Frage selbst sowie dem Erklärungstext der Frage. Des Weiteren werden hier die Antwortmöglichkeiten und Bilder definiert und gespeichert, in welcher Frage der Spieler sich gerade befindet.

Über den Inspektor können diese Daten angepasst werden, um das Quiz zu verändern oder zu erweitern. Dabei gibt es kein maximales Limit an möglichen Quiz-Fragen.

Die Verarbeitung dieser Informationen geschehen im Code des "QuizManager"-Scripts. Der momentanen Zustand des Spiels, also in welcher Frage der Spieler sich befindet und ob er sich vor, während oder nach einer Frage befindet, wird mit den Variablen "currentQuizIndex" und "currentState" festgehalten. Der State bestimmt, welches Panel im Canvas angezeigt wird und der Index, welche Frage angezeigt wird. Beim Anklicken einer Antwort wird der Button-Index mit der richtigen Antwort der Frage verglichen, welche ebenfalls im "QuizManager" hinterlegt wird.

Nach der letzten Frage findet ein Szenenwechsel zum nächsten Spiel statt.

Spiel 4: BASIC (Code Spiel)

Auch in diesem Spiel sind die wichtigsten Komponenten im Canvas. Für die verschiedenen Anzeigen sind hier wieder einzelne Panels erstellt worden. So gibt es das Panel "CodeGame" für die einzelnen Code-Blöcke, ein "IntroPanel" für den Erklärungstext und ein "OutroPanel" für den Abschlusstext. Über das Gameobject "CodeManager" und dem dazugehörigen Skript "CodeManager" wird die Logik des Spiels behandelt.

CodeManager

Damit die einzelnen Code-Blöcke per Drag & Drop beweglich sind, sind sie mit dem Skript "CodeBlocks" verbunden. Darin wird das Verschieben der Blöcke beschrieben sowie die Abfragen, ob ein Code-Block gerade über einer gültigen Stelle ist. Zeitgleich verhindert das Skript, dass die Blöcke aus dem Bildschirm geschoben werden können. Im Inspektor des Gameobject "CodeManager" sind die einzelnen Texte des Levels zu finden. Darunter der Intro- und Outro-Text, welche hier auch bearbeitet werden können. Die einzelnen Panel des Canvas werden wieder je nach aktuellem Stand des Spiel ein- oder ausgeblendet. Zusätzlich wird hier ermittelt, ob die Code-Blöcke vom Spieler in die richtige Reihenfolge gebracht wurden. Dafür wird die Methode "runCode" verwen-

det, die ausgeführt wird, sobald der Spieler auf den "RUN"-Button auf dem Display drückt. Je nach Reihenfolge der Blöcke werden die einzelnen Bilder, die die Münze darstellen, angezeigt. Anschließend wird die Reihenfolge mit einer fest eingestellten Reihenfolge verglichen, bei der die Münze richtig zusammengestellt ist. Dabei gibt es zwei Varianten, bei denen der Spieler ein richtiges Ergebnis bekommt, da die Reihenfolge der Striche auf der Münze keinen Unterschied macht. Bei einer richtigen Lösung wird das Panel des Outro-Texts angezeigt und anschließend zurück zum Hauptmenü weitergeleitet.

Hauptmenü: Menu

Die einzelnen Szenen sind über das Hauptmenü erreichbar. Für jede Scene ist hier ein eigener Button dargestellt, der über das Script "Menu" zu den anderen Szenen weiterleitet.

APK-Datei erstellen

Um das Unity Projekt auf einem Smartphones spielen zu können, muss daraus eine apk-Datei gemacht werden. Dafür kann unter dem Punkt "Build Profiles" die Plattform auf Android geändert werden und anschließend der Build-Prozess gestartet werden. Diese Datei kann nun auf einem Android-Smartphone installiert werden. Damit das funktioniert, muss bei einigen Smartphones der Entwicklermodus eingeschaltet sein.

Repository

Link zu Repository : <https://github.com/Ponysuppe/AuraDonator.git>

3.6 Aufgabenteilung

Silvan	Vanessa	Lennard
Level 1 Kippschalter-Spiel 3D-Modelle: Schalter	Level 3 Quiz-Spiel	Level 2 Zusammenbau-Spiel (Coding, UI, Mechaniken, VFX, Texte)
AR-Tracking, Wechseln zwischen Menu, Ar und Apple II	Level 4 Code-Spiel	Allgemeine Story und Verlauf der Schnitzeljagd
Story für Kippschalter + Übergang zu Apple II	3D-Modelle: CπRat & Apple II & UI-Elemente	Konvertierung der Dokumentation in LaTeX und Abgabe

3.7 Resümee und Ausblick

Rückblickend bot die Entwicklung der Schnitzeljagd-App wertvolle Erkenntnisse im Bereich der mobilen Augmented-Reality-Entwicklung sowie des Projektmanagements. Während der Entwicklung hat sich gezeigt, dass eine präzise Vorplanung und eine kontinuierliche Dokumentation unerlässlich waren, um die vier verschiedenen Spielmechaniken erfolgreich in Unity zusammenzuführen.

Eine der zentralen Lehren des Projekts betrifft die Definition von Anforderungen im Museums Kontext. Während der Entwicklungsphase wurde deutlich, dass getätigte Aussagen und Rahmenbedingungen bezüglich der Besucherinteraktionen vor Ort dynamisch sind und sich kurzzeitig ändern können.

Auf technischer Ebene stellte die kollaborative Arbeit mit Unity das Team vor Herausforderungen. Aus vorher abgeschlossenen Projekten hat das Team jedoch den Best Practice übernommen, dass zu jedem Zeitpunkt jeweils nur eine Person an einer Szene arbeitet. Dadurch wurde das Problem von häufigen unübersichtlichen Merge Konflikten verhindert.

Obwohl der finale Prototyp erfolgreich umgesetzt wurde, verbleiben für eine finale Produktionsreife der Anwendung noch einige offene Arbeitspakete. Die App ist aktuell ausschließlich auf Android-Geräten lauffähig. Ein primäres To-do ist daher die Portierung und Optimierung für iOS, um auch Apple-Geräte nativ zu unterstützen und eine breitere

Zielgruppe im Museum zu erreichen. Ebenfalls bietet das Hauptmenü noch Optimierungsmöglichkeiten, in dem beispielsweise ein Hinweis auf die Befolgung der korrekten Reihenfolge der Spiele für das beste Spielerlebnis notwendig ist.

Ebenfalls ist es im momentanen Zustand nicht möglich, zurück in die AR-Szenen zu wechseln, sobald das Quiz-Spiel einmal gestartet wurde. Dies liegt daran, dass in der Quiz-Szene kein additives Laden mehr genutzt wird und dadurch die Menu-, AR- und Apple II-Szenen gelöscht werden. Auf spielerischer Ebene bedarf die AR-Projektion des Apple II in der ersten Phase des Zusammenbauspiels ein paar kleine Anpassungen. Die Projektion ist momentan beispielsweise senkrecht platziert und flackert.

Über die Fertigstellung des aktuellen Funktionsumfangs hinaus bietet das Konzept der Schnitzeljagd großes Potenzial für zukünftige Erweiterungen. Um den Wiederspielwert für die Besucher zu erhöhen, könnte eine Level-Variation implementiert werden, sodass aufeinanderfolgende Durchgänge unterschiedliche Spielerfahrungen ermöglichen.

Darüber hinaus lässt sich das System skalierbar erweitern, indem weitere historische Exponate in die App integriert werden. Dies könnte man beispielsweise in Form weiterer Level in der Art des Zusammenbauspiels oder weiterer Fragen im Quizspiel realisieren. Langfristig könnte dadurch die Anwendung so zu einem festen, modularen Bestandteil des Museums werden.

Kapitel 4

Launcher App

4.1 Problem

Es hat sich im Laufe der Projektentwicklung bei mehreren AR-Projekten herausgestellt, dass die geplante Idee mit einer Webanwendung nicht möglich ist. Durch diese Gegebenheit sind unterschiedliche Ideen für die Bereitstellung für die Besucher*innen entstanden. Eine dieser Ideen war, dass vorne Tablets für die Besucher*innen herausgegeben werden können und diese Idee ist im folgenden weiter verfolgt worden.

4.2 Idee

Diese Idee besteht daraus, dass Tablets vorne an der Rezeption liegen, auf denen die hier entwickelten Projekte und auch zukünftige Projekte drauf installiert sind. Diese können die Besucher*innen vorne ausleihen und damit die XR interaktiven Elemente des Museums erleben. Wenn das Tablet angeschaltet wird, öffnet sich standartmäßig eine Launcher-App, welche für jedes Projekt einen Button anzeigt, um diese zu öffnen. Eine Bedingung ist, dass jede App einen Weg liefern muss, zurück zur Launcher-App zu kommen.

4.3 Projekt

Im Rahmen dieser Idee, ist ein Proof of concept mit der OCMExplore-Gruppe entstanden. Dieses bietet einen Button in der Mitte des Bildschirms, mit welchem die OCMExplore-App geöffnet werden kann.

4.4 Aufgabeneinteilung

Dieses Projekt ist von Louisa Valerie Müller entwickelt worden.

4.5 Weitere Überlegungen

Im Laufe der Entwicklung des Proof of concepts sind weitere Überlegungen entstanden.

4.5.1 Modularität

Dadurch, dass die technische Grundlage rein programmativer Natur ist, bietet sich die Möglichkeit ein Adminpanel einzubinden, in welchem weitere Projekte eingebunden werden können. Dies würde die programmatische Wartungsarbeit stark reduzieren.

4.5.2 Vor-Ort Umsetzung

Die Umsetzung vor Ort bestünde aus Android-Tablets, welche XR-Kompatibilität bieten. Dazu kann man dann spezialisiert auf diese Tablets entwickeln, welche genauere Voraussetzungen dann noch erstellen würde. Diese Tablets sind so eingerichtet, dass die Besucher:innen nur auf diese Launcher-App zugreifen können. Dies könnte mit einer App, wie zum Beispiel, MacroDroid umgesetzt werden. Vorne an der Rezeption werden Tablets verliehen und auch geladen. Genauere Konfiguration von Android muss weiter ausprobiert werden. Für die Zuordnung von App und Ort im Museum könnte ein Iconsystem dienen, wo für jede App ein Icon existiert und diese dann bei den jeweiligen Stationen platziert sind.

4.5.3 Design

Das Design der App richtet sich nach dem Stil des OCM's. Genaueres steht im technischen Bericht.

4.6 Fazit

Die Idee einer allgemeinen Launcher App ist technisch umsetzbar und von diesem Proof of concept aus weiter entwickelbar.